

ComicCraft AI: AI-Powered Comic Story Creator

Project Description

ComicCraft is a web-based AI application that transforms a user-provided story idea into a structured multi-panel comic. Users enter a story prompt together with character, setting, tone, and art-style details. The application generates a panel-by-panel outline, narration and dialogue, AI-generated comic illustrations, a preview, and a downloadable PDF.

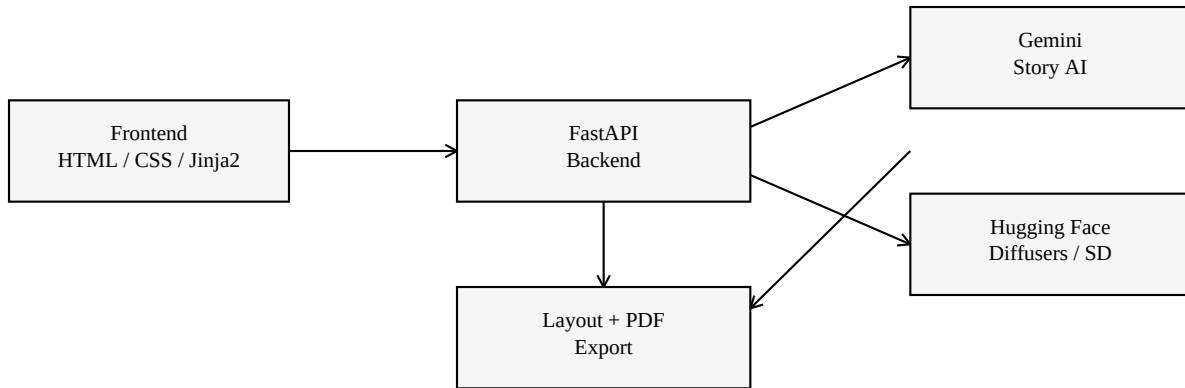
Scenarios

Scenario 1: A user enters a short story idea and receives a five-panel comic with generated story content and visuals.

Scenario 2: A user changes the character, setting, tone, or art style and regenerates the comic to create a different creative result.

Scenario 3: After reviewing the generated panels, the user downloads the completed comic as a PDF for saving, printing, or sharing.

Technical Architecture



Description

ComicCraft uses a three-part architecture: an HTML/CSS/Jinja2 frontend, a FastAPI backend, and AI services. The frontend collects story inputs and displays the generated comic. FastAPI handles routing, validation, orchestration, layout building, and PDF export. Google Gemini services generate structured story content, while Hugging Face Diffusers/Stable Diffusion generates comic-style illustrations.

Database Note

No database component is documented in the available ComicCraft architecture. Therefore, an ER diagram is not applicable to the documented implementation.

Pre-requisites

- Python programming and package management
- FastAPI and Uvicorn
- HTML, CSS and Jinja2 templates
- Google Gemini / Google GenAI API familiarity
- Hugging Face Diffusers and Stable Diffusion
- Pillow and PDF generation with FPDF/fpdf2
- Basic Git/version-control knowledge

Project Workflow

Milestone 1: Model Selection and Architecture

- Activity 1.1: Research and select generative AI models.
- Activity 1.2: Define frontend, backend, and AI architecture.
- Activity 1.3: Set up the Python development environment.

Milestone 2: Core Functionalities Development

- Activity 2.1: Develop outline, story, image, layout, and PDF generation functions.
- Activity 2.2: Integrate the functions through FastAPI routes.

Milestone 3: routes.py Development

- Activity 3.1: Implement and connect the main application routes and error handling.

Milestone 4: Frontend Development

- Activity 4.1: Develop the user input and comic preview interfaces.
- Activity 4.2: Integrate FastAPI with Jinja2 dynamic templates.

Milestone 5: Deployment and Testing

- Activity 5.1: Prepare local deployment.
- Activity 5.2: Test the complete generation and PDF-export workflow.

Milestone 6: Completion

- Application tested locally, five-panel comic generated, and PDF export flow verified.

Milestone 1: Model Selection and Architecture

Overview

This milestone focuses on selecting AI models suitable for structured comic outlines, detailed narration/dialogue, and comic-style illustrations. The documented implementation combines Google Gemini models for story generation with Stable Diffusion for image generation.

Activity 1.1: Research and Select the Appropriate Generative AI Model

Project requirements:

- Structured panel-by-panel comic outline
- Full narration and character dialogue
- Visual illustration for every panel

Documented model selection:

- Gemini Flash — structured five-panel outline generation.
- Gemini Pro — detailed character narration and dialogue.
- Stable Diffusion — comic-style illustrations from image prompts.

Activity 1.1: AI Integration and Configuration

Model Integration

The AI pipeline is sequential. The outline generator first creates a structured five-panel plan. The story generator then expands the outline into narration and dialogue. The image generator creates an illustration for each panel from the corresponding image prompt. The resulting data is passed to the layout and export stages.

Environment Variables

Sensitive service credentials are kept outside the source code through environment variables/.env configuration. The documented project uses a Gemini API key and a Hugging Face API token for the AI services.

Completion

YES — AI model integration and the end-to-end generation workflow were completed and tested in the local application.

Activity 1.2: Define the Architecture of the Application

Architecture Overview

Frontend: HTML, CSS and Jinja2 templates collect the story prompt, character name, setting, tone and art style and display the generated comic.

Backend: FastAPI receives form/JSON input, coordinates generation, handles errors, builds the layout and exposes the export workflow.

AI Integration: Google Gemini generates the structured story content; Hugging Face Diffusers/Stable Diffusion generates the panel illustrations.

Templates

index.html collects user inputs; comic_preview.html displays the generated comic; export_success.html confirms the export flow.

Activity 1.3: Development Environment

Python, FastAPI, Uvicorn, Jinja2, python-multipart, Google GenAI, Diffusers, Transformers, Accelerate, Torch, Pillow, fpdf2 and related dependencies are used in the project.

Activity 1.3: Set Up the Development Environment

Environment Setup

A Python environment is used to isolate project dependencies. The application is started from the ComicCraftAI root directory using the FastAPI/Uvicorn application entry point.

Local Server

The verified local application runs at:
<http://127.0.0.1:8000>

FastAPI documentation is available at:
<http://127.0.0.1:8000/docs>

Project Structure

The repository contains the app backend and templates, with requirements.txt defining the application dependencies. The working project is named ComicCraftAI.

Completion

YES — Development environment and local server setup were completed.

Milestone 2: Core Functionalities Development

Activity 2.1: Develop the Core Functionalities

- `generate_outline()` creates a structured five-panel outline.
- `generate_story()` expands the outline into narration and dialogue.
- `generate_image()` creates the visual illustration for each panel.
- `build_comic_layout()` matches images with panel content.
- `save_pdf()` compiles the comic into an exportable PDF.

Activity 2.2: Implement the FastAPI Backend

FastAPI routes accept story prompt, character name, setting, tone and art style. The backend passes the data through the AI pipeline, prepares the final layout, renders the preview page and provides the PDF export flow.

Milestone 3: routes.py Development

Overview

The routes.py module coordinates the main ComicCraft workflow. It connects user requests with the outline, story, image, layout and export functions and provides appropriate error handling.

Activity 3.1: Main Application Routes

- / — loads the homepage.
- /generate — processes form input and generates the comic preview.
- /generate-comic/json — accepts JSON input and returns generation/layout information for API clients.
- /export-success — displays export confirmation.
- /test-image — tests image generation independently.

Activity 3.1: Comic Generation Route

Generation Sequence

For a normal comic-generation request:

1. Receive story prompt and optional character/setting/tone/style values.
2. Generate a structured five-panel outline.
3. Generate narration and dialogue.
4. Generate an image for each panel.
5. Build the final comic layout.
6. Render the preview page.
7. Export the comic to PDF when requested.

Error Handling

The route handlers use validation and try/except handling so API errors, JSON issues and unexpected generation failures can be handled without breaking the overall application flow.

AI Generation and Prompt Processing

Outline Generation

Gemini Flash is used for structured panel planning. Each panel contains a panel number, title, scene description and image-generation prompt.

Story Generation

Gemini Pro is used to expand the outline into full comic narration and character dialogue.

Image Generation

Stable Diffusion through the Hugging Face Diffusers pipeline generates comic-style illustrations from the panel image prompts.

Integration

AI responses are cleaned and passed into the Jinja2 templates for preview and into the layout/export functions for the final PDF.

AI Response Integration and Validation

Pipeline Validation

The backend verifies that the generated content can be organized into the expected panel structure before the preview/export stages. The layout builder associates each generated image with its corresponding panel story.

API Interfaces

The application supports both browser form submission and a JSON-based comic-generation endpoint. This provides a direct API path in addition to the normal web workflow.

Output

The final result contains the generated comic panels, associated story content, and an exportable PDF path/file.

Milestone 4: Frontend Development

Activity 4.1: Designing and Developing the User Interface

The main page provides a simple form for Story Prompt, Main Character Name, Setting, Story Tone and Art Style. The interface uses HTML/CSS and Jinja2 and provides a direct flow from user input to comic generation.

Comic Preview

The preview page displays the generated panels sequentially. Each panel contains a title, AI-generated image, scene description, captions/narration and image-prompt reference where provided.

Activity 4.2: Creating Dynamic Templates with FastAPI's Jinja2

Template Binding

FastAPI's Jinja2Templates binds backend data to the HTML pages:

- / renders index.html.
- /generate renders comic_preview.html.
- /export-success renders export_success.html.

Dynamic Rendering

The preview template loops through the generated layout and displays the panel number, title, image, scene description, captions and narration. This keeps the generated comic content synchronized with the backend output.

Completion

YES — Frontend pages and dynamic template rendering were completed.

Milestone 5: Local Deployment

Activity 5.1: Preparing the Application for Local Deployment

The project is run from the ComicCraftAI root directory using the FastAPI application entry point. Required Python packages are installed from requirements.txt and environment variables are configured before starting the server.

Server

Verified application endpoint:
`http://127.0.0.1:8000`

The application successfully reached the ComicCraft AI Creator Studio interface.

Activity 5.2: Testing and Verifying Local Deployment

Verification

The complete local workflow was tested:

- Story setup page opened successfully.
- User inputs were accepted.
- Comic generation completed.
- A five-panel comic was produced.
- Generated comic result/preview page opened successfully.
- PDF download control was displayed.

Result

YES — Local application testing and the end-to-end comic generation flow were completed.

Project Hosting and Demonstration

Source Repository

<https://github.com/mvishal6002-maker/ComicCraftAi>

Demo / Project Files

<https://drive.google.com/drive/folders/1NDLCLIYRRdQ1kjjxbd1fUFZ3-hLjxxTX>

Hosted / Demonstration Status

YES — Source repository and project demonstration/reference location were provided.

Project Completion Status

YES

Final Testing and Completion

Functional Completion

YES — The application was successfully run locally and a five-panel comic was generated from the Creator Studio interface.

Export Completion

YES — The generated result page displayed the **DOWNLOAD COMIC PDF** control and the project documentation describes the PDF export flow.

Project Dates

Start Date: 25/09/2026

End Date: 30/09/2026

Team ID

6ab3ecd3a238dc999eb3fcc2

Exploring the Website's Web Pages

Home / Creator Studio Page

The ComicCraft homepage is the starting point for creating an AI-generated comic. It provides the story setup form and a page preview area. Users can enter the story prompt, character name, setting, tone and art style before generating the comic.

Generation Flow

After submission, the backend runs the AI generation pipeline and returns the generated comic preview. The working local interface was verified at 127.0.0.1:8000.

Comic Preview and Export

Input Section

The Story Setup section accepts the story prompt and optional character, setting, tone and art-style information. The Generate Comic action starts the AI workflow.

Results Section

The finished-draft page presents the generated comic and indicates the number of panels. In the completed test, the result contained 5 panels and displayed a DOWNLOAD COMIC PDF button.

User Outcome

The workflow provides an end-to-end path from a short idea to a visual comic preview and downloadable PDF output.

Conclusion

Project Conclusion

ComicCraft demonstrates an end-to-end AI-powered comic creation workflow. It combines Gemini-based story generation with AI image generation to transform a short user idea into a structured multi-panel comic with narration, dialogue, visuals and PDF export.

Outcome

The application was successfully executed locally, a five-panel comic was generated, and the final-draft page with PDF download was reached. The project is completed based on the available project documentation and verification screenshots.

Completion Status

YES — PROJECT COMPLETED

Team ID

6ab3ecd3a238dc999eb3fcc2